Image Representation Comparison

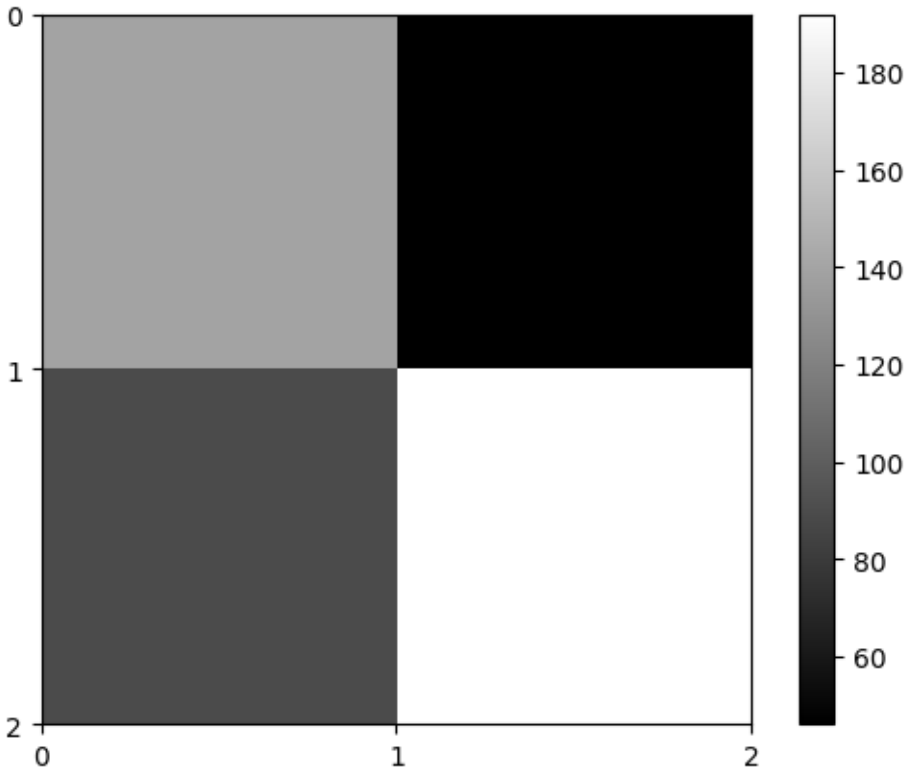
Marco Di Biasi

Classical Image representation

In the classical representation, a black and white image is a matrix of integers, where every position represent a pixel, and the integer represent the color intensity.

[1]:
```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
```

[2]:
```
img = Image.open("test6.bmp").convert('L')
plt.imshow(img, cmap="gray", extent=[0, 2, 2, 0])
plt.xticks(range(0, 3))
plt.yticks(range(0, 3))
plt.colorbar()
plt.show()
```



The numpy array behind shows the image-encoding for classical computation

[3]:
```
print(np.array(img))
```

```
[[140  46]
 [ 89 192]]
```

Novel Enhanced Quantum Representation (NEQR)

The idea behind this approach for a quantistic image representation is to optimise memory by encoding all the the pixel into one single quantum register, taking advantage of the superposition property.

$$|I\rangle = \frac{1}{2^n} \sum_{Y=0}^{2^n-1} \sum_{X=0}^{2^n-1} |f(Y,X)\rangle \otimes |YX\rangle$$

Where:
- $|f(Y,X)\rangle$ represents the grayscale intensity, also written as $|C^i_{YX}\rangle$ - $|YX\rangle$ represent the pixel position

The Quantum Circuit creates a **superposition of the positional and color registers** for a $2^n \times 2^n$ image.

NEQR: Code Implementation

Image preparation

By first, we create an array of bits by normalizing and converting the oginal matrix.

[4]:
```
bit_matrix_img = np.vectorize(lambda x: format(x, "02b"))(np.array(img) // 64)
```

[5]:
```
print(bit_matrix_img)
```

```
[['10' '00']
 ['01' '11']]
```

To calculate the Qubits needed for the circuit the general formula is $q + 2n$ to represent a $2^n \times 2^n$ image with gray range 2^q.
In this case, we use a **2-bit color representation**, for 4 different levels of gray intensity.

[6]:
```
img_width = len(bit_matrix_img) #since the image is square, width and height␣
  ↪are the same
n_positions = int(np.ceil(np.log2(img_width))*2)
q = int(np.log2(4))

n_qubits = n_positions + q
```

[7]:
```
print(n_qubits, n_positions, q, sep=', ')
```

```
4, 2, 2
```

First step

To initialise the Quantum Circuit, all the Qubits must be set to $|0\rangle$:

$$|\psi\rangle_0 = |0\rangle^{\otimes 2n+q}$$

[8]:
```
from qiskit import QuantumCircuit
```

[9]:
```
qc = QuantumCircuit(n_qubits)
initial_state = np.zeros(2**n_qubits)
initial_state[0] = 1

qc.initialize(initial_state, range(n_qubits));
```

After the first step, the Quantum State is as follows:

$$|\psi\rangle_0 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

In this specific case, the image is $2px \times 2px$, so the state has $2 + 2\log_2 2 = 4$ Qubits.

Second step

The position register is brought into a superposition state while the color registers remain unaltered. Formally, U_1 is applied, such that:

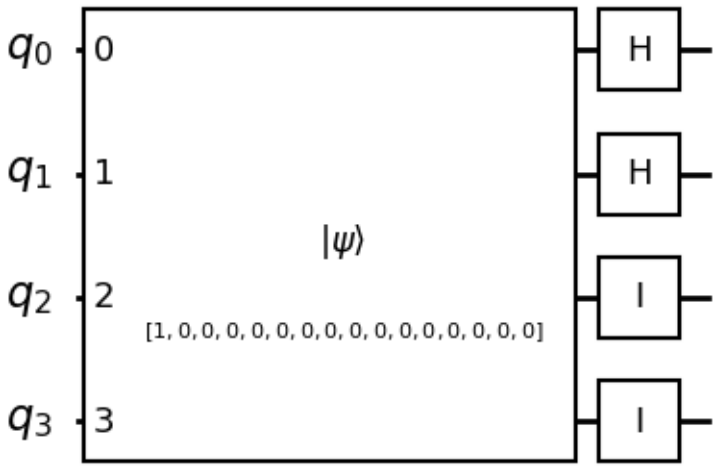
$$U_1 = I^{\otimes q} \otimes H^{\otimes 2n}$$

The Quantum state after this operation can be written as:

$$|\psi\rangle_1 = \frac{1}{2^n} \sum_{Y=0}^{2^n-1} \sum_{X=0}^{2^n-1} |0\rangle^{\otimes q} |XY\rangle$$

[10]:
```
for i in range(n_positions):
    qc.h(i)

for i in range(n_positions, n_qubits):
    qc.id(i)

qc.draw('mpl', style='bw')
```

[10]:



In a probabilistic interpretation, the state can collapse with equal probability onto one of the position Qubits.

Third step

The final step consists in setting the greyscale value for each pixel. Formally, U_{YX} is applied, such that:

$$U_{YX} = \Big(I \otimes \sum_{j=0}^{2^n-1} \sum_{i=0, ji \neq YX}^{2^n-1} |ji\rangle\langle ji|\Big) + \Omega_{YX} \otimes |YX\rangle\langle YX|$$

Detailed Analysis of the Operator U_{YX}

The operator U_{YX} can be decomposed into two different sub-operations to better understand its role. The first sub-operation, U'_{YX}, is written as follows:

$$U'_{YX} = \Big(I \otimes \sum_{j=0}^{2^n-1} \sum_{i=0,\ ji \neq YX}^{2^n-1} |ji\rangle\langle ji|\Big)$$

This part handles all the pixels exepts for YX, leaving their quantum states unaffected. Inside the summation, we have a projection operator $|ji\rangle\langle ji|$, which, when applied to $|ji\rangle$, results in $|ji\rangle$. If applied to any other state, it gives 0. Formally:

$$|ji\rangle\langle ji||ab\rangle = \begin{cases} |ji\rangle & if\ ab = ji \\ 0 & if\ ab \neq ji \end{cases}$$

The second sub-operation, U''_{YX}, is written as follows:

$$U''_{YX} = \Omega_{YX} \otimes |YX\rangle\langle YX|$$

The projection operator ensures that the greyscale transformation is applied only to the specific pixel (Y, X). The operator Ω_{YX} is the quantum operation designed for setting the greyscale levels for pixel (Y, X):

$$\Omega_{YX} = \bigotimes_{i=0}^{q-1} \Omega^i_{YX}$$

With Ω^i_{YX} quantum oracles:

$$\Omega^i_{YX} : |0\rangle \rightarrow \left| 0 \oplus C^i_{YX} \right\rangle$$

When the operator is applied to the initial quantum state, the result is:

$$U_{YX}(|\psi\rangle_1) = \frac{1}{2^n}\Big(\sum_{j=0}^{2^n-1} \sum_{i=0,\, ji \neq YX}^{2^n-1} |0\rangle^{\otimes q}|ji\rangle + |f(Y,X)\rangle|YX\rangle \Big)$$

Since the operator U_{YX} corresponds to a specific position, to determine the quantum state $|\psi\rangle_2$, U_2 is introduced, such that:

$$U_2 = \prod_{Y=0}^{2^n-1} \prod_{X=0}^{2^n-1} U_{YX}$$

Thus, the final Quantum State is:

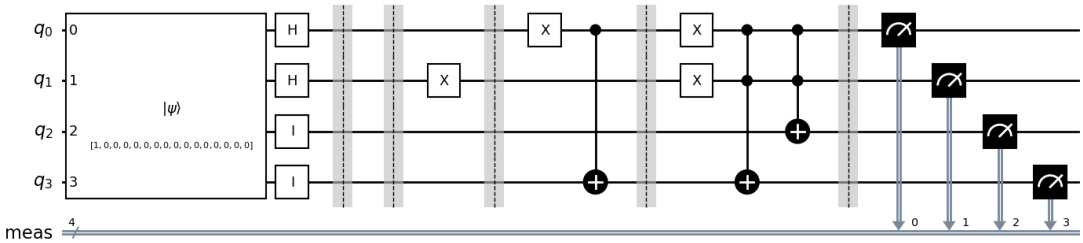
$$|\psi\rangle_2 = U_2(|\psi\rangle_1) = \frac{1}{2^n}\Big(\sum_{Y=0}^{2^n-1} \sum_{X=0}^{2^n-1} |f(Y,X)\rangle|YX\rangle \Big)$$

[11]:
```
h, w = img_width, img_width

for Y in range(h):
    for X in range(w):
        qc.barrier()
        bin_Y = bin(Y)[2:].zfill(int(np.log2(h)))
        bin_X = bin(X)[2:].zfill(int(np.log2(w)))
        pos_bin =  bin_Y + bin_X

        for i, bit in enumerate(pos_bin):
           if bit == '1':
                qc.x(i)

        f_YX = bit_matrix_img[Y,X]
```

```
        control_qubits = [i for i, bit in enumerate(pos_bin) if bit == '1']
        for i, bit in enumerate(f_YX[::-1]):
            if bit == '1' and control_qubits != []:
                qc.mcx(control_qubits, n_qubits-i-1)
```

[12]:
```
qc.measure_all()
qc.draw('mpl', style='bw', fold=35)
```

[12]:



The final encoding

$$|\psi\rangle_2 = \frac{1}{2^2}\Big(|00\rangle|10\rangle + |01\rangle|00\rangle + |10\rangle|01\rangle + |11\rangle|11\rangle\Big)$$

Getting the counts

[13]:
```
from qiskit_aer import Aer
from qiskit import transpile
```

[14]:
```
simulator = Aer.get_backend('qasm_simulator')
transpiled_circuit = transpile(qc, simulator)
job = simulator.run(transpiled_circuit, shots=10000)
res = job.result()
counts = res.get_counts()
```

[15]:
```
print(counts, '\n\n', bit_matrix_img)
#c0 c1 are the grey code
#c2 c3 are Y X
```

```
{'1010': 2475, '1000': 2448, '1111': 2588, '0001': 2489}

 [['10' '00']
 ['01' '11']]
```

Are there any advantages?

In this case the image has taken $2 \times 2 \times 2 = 8$bits, compared to the 4qubits we used. We now compare the formulas:

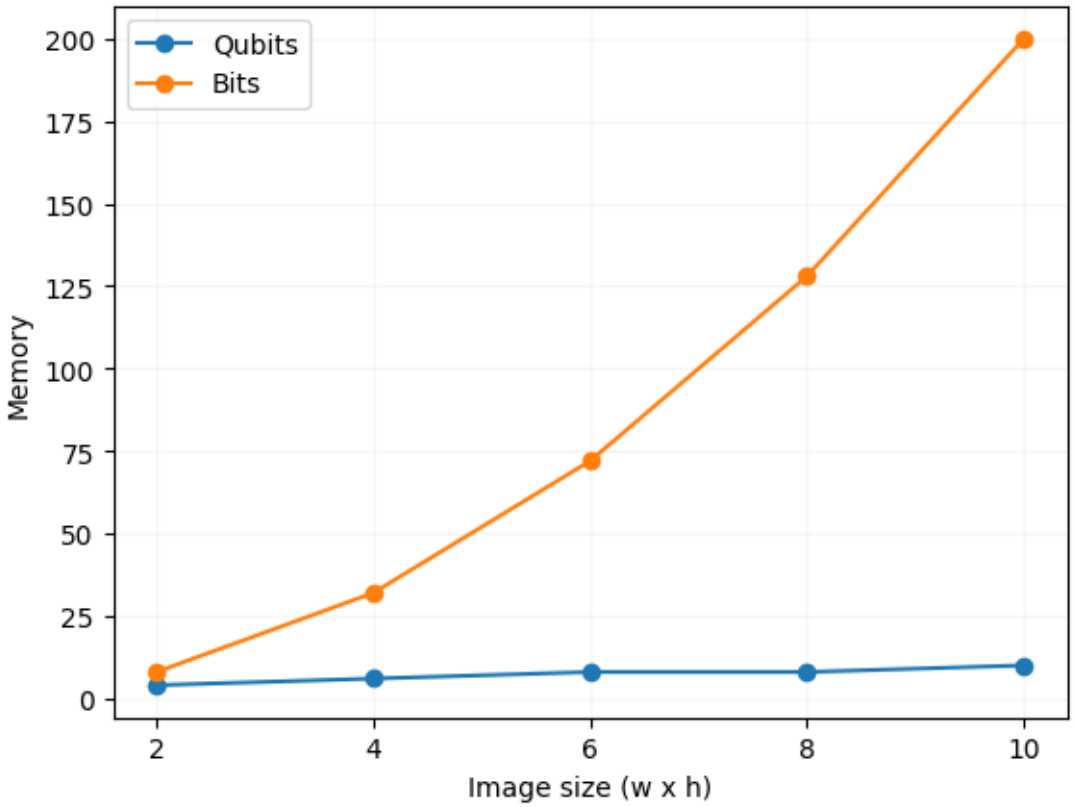
$$\text{Bits} = w \times h \times q$$

$$\text{Qubits} = q + 2n \qquad \text{with} \qquad n = \lceil \log_2(w) \rceil = \lceil \log_2(h) \rceil$$

Making the assumptions that we only operate with square images and with q fixed, we can plot the memory usage.

[16]:
```
q = 2
wh_values = np.arange(2, 11, 2)
n_qubits = q + 2 * np.ceil(np.log2(wh_values)).astype(int)
n_bits = q * wh_values * wh_values
```

[17]:
```
plt.plot(wh_values, n_qubits, marker='o')
plt.plot(wh_values, n_bits, marker='o')
plt.grid(alpha=0.1)
plt.xlabel("Image size (w x h)")
plt.ylabel("Memory")
plt.xticks(wh_values)
plt.legend(['Qubits', 'Bits'])
plt.show()
```



It is already noticeble how the number of qubits grows linearly, against the exponential growht of the bits. With $q = 2$, $h = w = 8$

$$\text{Bits} = 8 \times 8 \times 2 = 128$$

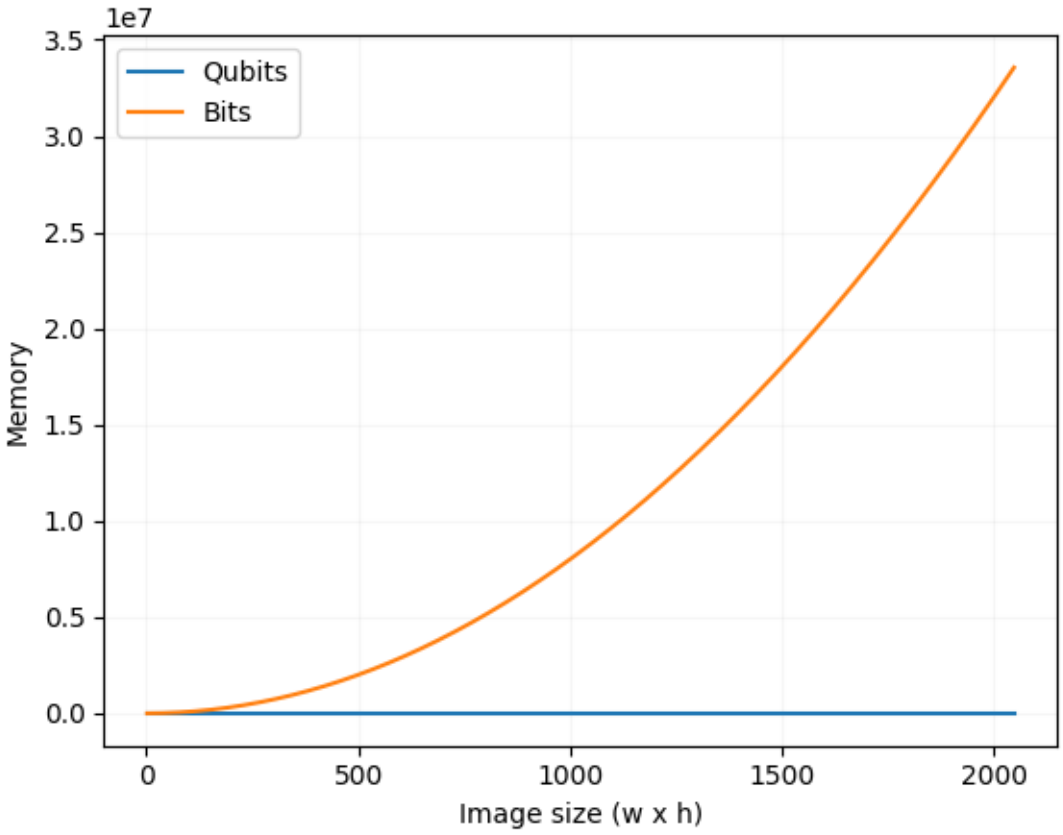
$$\text{Qubits} = 2 + 2\log_2 8 = 8$$

Let's try with real numbers!

For a black and white high definition square image: $q = 8$, $h = w = 2048$

[18]:
```
q2 = 8
wh2_values = np.arange(2, 2049, 2)
n2_qubits = q2 + 2 * np.ceil(np.log2(wh2_values)).astype(int)
n2_bits = q2 * wh2_values * wh2_values
```

[19]:
```
plt.plot(wh2_values, n2_qubits)
plt.plot(wh2_values, n2_bits)
plt.grid(alpha=0.1)
plt.xlabel("Image size (w x h)")
plt.ylabel("Memory")
plt.legend(['Qubits', 'Bits'])
plt.show()
```



$$\text{Bits} = 2048 \times 2048 \times 8 = 33.226.752$$

$$\text{Qubits} = 8 + 2\log_2(2048) = 30$$

Conclusion

Beyond the advantage in memory usage without any compression algorithm, the Novel Enhanced Quantum Representation can be convenient in other aspects. However, like any technique, it may also have its drawbacks, which should be carefully considered depending on the specific application.

Filtering and Transforming

In Classical Computation, a standard transformation can be performed with a time complexity of $O(N)$, by iterating along the full matrix. However, by taking advantage of superposition, we can potentially perform the same operation with a time complexity of $O(logN)$, achieving an exponential improvement. But the real advantages emerge when we apply a filter. In classical computation, applying a filter by convolution takes $O(KN)$, with K being the size of the filter. It is evident that with big filters, the complexity approaches $O(N^2)$. Thanks to the Convolution Theorem, we can reduce this complexity to $O(NlogN)$, using the Fast Fourier Trasform. With Quantum Computation, we can take it a step further. The performance of a Quantum Fourier Transform, without considering the measurement phase, takes $O(log^2N)$, thanks to the quantum parallelism.

Pixel Access

While every access in the Classical Computation takes $O(1)$ without affecting the other pixels, in the NEQR, informations are stored differently. Each individual access collpases the entire quantum state. Therefore, it is necessary to measure the system to get all the position-color pairs.

In conclusion, while the theoretical advantages of NEQR and quantum computation are fascinating, practical considerations must be addressed before they can be widely applied. With advances in quantum hardware and error correction, these methods may become more viable for specific applications, but for now, classical techniques remain highly effective for many real-world scenarios.

References

1. Zhang, Y., Lu, K., Gao, Y., & Wang, M. (2013). *NEQR: A novel enhanced quantum representation of digital images*. ResearchGate. Retrieved from https://www.researchgate.net/publication/257641933_NEQR_A_novel_enhanced_quantum_representation_of_digital_images
2. Musk, D. (2020). *A comparison of quantum and traditional Fourier transform computations*. Retrieved from https://d197for5662m48.cloudfront.net/documents/publicationstatus/53281/preprint_pdf/b34ed0bc05160ab903a3790ad0dbe450.pdf
3. Qiskit. (2023). Image processing with FRQI and NEQR [Notebook]. GitHub. https://github.com/Qiskit/textbook/blob/main/notebooks/ch-applications/image-processing-frqi-neqr.ipynb?short_path=6d10247
4. Stanco, F. (2024). Convoluzioni e Fourier [PowerPoint slides]. Corso di Interazione e Multimedia e Laboratorio A-L, Università di Catania.